DURHAM COLLEGE | DCCONNECT

Assignment 4 - JavaScript

Course: 202642.11745-INFT-1206-05 - Web Development - Fundamentals

Criteria	**Level 4** 3 points	**Level 3** 2 points
Submission Requirements	Clickable links were provided in DC Connect drop box to all files for the assignment	Clickable links were prov
HTML5 Validation	Submitted HTML file(s) include (at the top of the file): <!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"> and when run through an HTML5 validator returns zero (0) errors across all files (i.e. files are all HTML5 valid)	Submitted HTML file(s) <!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8 and when run through

HTML comments	All published files had HTML comments that conformed to the requirements
	
	<!--
	        Name: Your Name
	        File: filename.html
	        Date: XX September 202X
	        A very brief description of the file.
	-->
		Most published files add
		
		<!--
		        Name: Your Name
		        File: filename.ht
		        Date: XX Septembe
		        A very brief desc
		-->

Criteria	**Level 4** 3 points	**Level 3** 2 points
GitHub Requirements	All published files had at least 5 commits including meaningful descriptions of the changes of being committed	Most published files had

Criteria	**Level 4** 6 points	**Level 3** 4 points	**Level 2** 2 points	**Level 1** 0 points	**Criterion Score**

Criteria	**Level 4** 6 points	**Level 3** 4 points	**Level 2** 2 points	**Level 1** 0 points	**Criterion Score**
Silly Story Generator Requirements	The .html/.js file(s) published implemented all of the following requirements • created a main.js file that was implemented by your *.html file • created a storyText variable and insertX, insertY, and insertZ arrays from the provided files in main.js • copied the provided event handler and incomplete function in main.js • in result() function creates a new variable newStory set equal to storyText • in result() function creates three (3) variables xitem, yItem and zItem that are respectively set by selecting a random element from insertX, insertY and insertZ using the randowmValueFromArray() function • in result() function, uses a newStory.replaceAll() function call to replace :insertx:, :inserty: and :insertz: with the random insertX, insertY and insertZ variables • in result function(), if a user enters a name in the cutomname text input, replacces "Bob" in the newstory variable with the name entered • in result function(), if the "uk" radio input is selected, converts the 300lb to its equivalent in "stones" • in result function(), if the "uk" radio input is selected, converts the 94 degrees fahrenhiet to its equivalent in centigrade using the equation celsius = (fahrenheit-32) * 5 / 9 • in result function(), sets the story.textContent equal to the modified newStory variable	Files published implemented most of the requirements (>75%)	Files published implemented some of the requirements (>50%)	No or very few of the required changes were implemented to the published file(s) (<50%)	/ 6

<table>
<tr><th>Criteria</th><th>Level 4
6 points</th><th>Level 3
4 points</th><th>Level 2
2 points</th><th>Level 1
0 points</th><th>Criterion Score</th></tr>
<tr><td>Image gallery Requirements</td><td>The .html/.js file(s) published implemented all of the following requirements
include the external JavaScript main.js file using a <script src="filename.js"> </script> tag.in index.html
creates an array of the image file names, including the const keyword (to make the array constant/immutable)
in main.js. loops through the array of images starting at 1 (that is included in the file name of the first picture, not 0 the index of the first picture in the array)
in main.js, adds an event listener for the "click" event on all thumbnails
in main.js, includes a handler that will lighten/darken the button depending current value of const btnClass = btn.getAttribute('class'); i.e. if it is "dark" lighten the image, if it is "light" (or anything oterh than "dark") lighten the image</td><td>Files published implemented most of the requirements (>75%)</td><td>Files published implemented some of the requirements (>50%)</td><td>No or very few of the required changes were implemented to the published file(s) (<50%)</td><td>/ 6</td></tr>
</table>

Criteria	**Level 4** 6 points	**Level 3** 4 points	**Level 2** 2 points	**Level 1** 0 points	**Criterion Score**
Object building practice Requirements (creating the Ball class)	The main.js file published implements all of the following requirements • implements the four (4) provided constants for: canvas; ctx; width; and, height • creates a Ball class with a constructor that takes 5 arguments: x, y, velX, velY, color, size that sets the Ball object's attributes to the passed values • implements the provided draw() function • implements the provided update() function • implements the while() loop that creates 25 random balls • implements the provided loop() function • implements the provided collision-Detect() function that will allow collision detection • included a collsion-Detect() function call in the loop() function	Files published implemented most of the requirements (>75%)	Files published implemented some of the requirements (>50%)	No or very few of the required changes were implemented to the published file(s) (<50%)	/ 6

Criteria	**Level 4** 6 points	**Level 3** 4 points	**Level 2** 2 points	**Level 1** 0 points	**Criterion Score**
Adding bouncing balls features Requirements	The main.js file(s) published implements all of the following requirements • creates a new Shape class that is a parent of the Ball class (created in last project) • create that accepts four (4) arguments: x, y, velX, velY • modifies the existing Ball constructor, replacing the first four (4) argument sets to a super() constructor call and sets the color and size attributes in the Ball constructor • creates a new EvilCircle class that is a child of the Shape class • implements in the EvilCircle class a method named draw() that complies with the requirements • implements in the EvilCircle class a method named checkBounds() that complies with the requirements • implements in the EvilCircle class a method named collisionDetect() that complies with the requirements • creates and displays an EvilBall oject each time the loop() function is called	Files published implemented most of the requirements (>75%)	Files published implemented some of the requirements (>50%)	No or very few of the required changes were implemented to the published file(s) (<50%)	/ 6

<table>
<tr><th>Criteria</th><th>Level 4
6 points</th><th>Level 3
4 points</th><th>Level 2
2 points</th><th>Level 1
0 points</th><th>Criterion Score</th></tr>
<tr>
<td>Structuring a feedback form Requirements</td>
<td>The index.html file published implements all of the following requirements:

Implementing form controls

Convert first two "Facilities" line groups into radio buttons with labels and a fieldset legend.
Set the first radio button in each of those groups as the default selected option.
Convert the third "Facilities" line group into checkboxes with labels and a fieldset legend.
Convert both line groups in "About your hosts" into labeled select (drop-down) menus.
In "Any other feedback?":
Add a multi-line textarea.
Use the existing text as its label.
In "Your details":
Add three appropriate text-input fields.
Turn each existing line into its label.
Convert the word "Submit" into a submit button.

<hr>
Structuring the form

Wrap the entire form in the correct form element.
Add wrappers around each form section with class <code>form-section</code>; remove placeholder <code>--</code>.
Add wrapper elements around label/control pairs to place each on
</td>
<td>Files published implemented most of the requirements (>75%)</td>
<td>Files published implemented some of the requirements (>50%)</td>
<td>No or very few of the required changes were implemented to the published file(s) (<50%)</td>
<td>/ 6</td>
</tr>
</table>

Criteria	Level 4 6 points	Level 3 4 points	Level 2 2 points	Level 1 0 points	Criterion Score
	its own line (class `separator`). • Insert a line break between the textarea and its label. **Additional HTML features** • Place the provided CSS into an external `*.css` file and `<link>` it into the `index.html` file • Mark up headings properly: ◦ Top-level: "We want your feedback!" ◦ Second-level: "Facilities", "About your hosts", "Any other feedback?", "Your details" • Mark up the introductory paragraph. • Turn "little house in the woods" and "prize draw" into anchor links using `href="#"`. • Insert decorative wide image below intro paragraph: ◦ Source: provided woodland image URL ◦ Empty alt text (decorative image)				

Total	/ 42

Overall Score

Level 4	Level 3	Level 2	Level 1
6 points minimum	4 points minimum	2 points minimum	0 points minimum